

Program Transformations

Outline

- Java Bytecode Method Calls
- Abstract Syntax Trees
- Program Transformations
 - Bytecode
 - AST

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w        #3      // double 2.0d
```

```
    5: invokestatic  #5      // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w        #3      // double 2.0d
```

```
   13: invokestatic  #5      // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic  #6      // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Constant Pool

Table that stores various constants needed for execution

Includes:

- Literals
- Symbolic references: accessing an object in memory using its name (or identifier) rather than its direct memory address

`invokestatic #2` executes the static method at the second index of the constant pool

Number sign refers to the constant pool

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w           #3      // double 2.0d
```

```
    5: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w           #3      // double 2.0d
```

```
   13: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic    #6      // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Non-static method calls

Method calls:

`invokevirtual` instead of `invokestatic`

Constructor calls (object creations):

`invokespecial`

Method Calls - nonstatic

```
class Example {  
    public void greet() {  
        System.out.println("hello");  
    }  
}  
  
Example e = new Example();  
example.greet();
```

```
0: new          #2          // Create a new Example object  
3: dup          // Duplicate the reference to the object on the operand stack  
4: invokespecial #3          // Call constructor (Example.<init>)  
7: aload_1      // Load the reference to the Example object (example)  
8: invokevirtual #4          // Call greet() on the instance of Example  
11: return
```

Given this source, what is the bytecode?

```
public class CountdownTimer {
    public static void main(String[] args) {
        CountdownTimer timer = new CountdownTimer();
        timer.startCountdown(10);
    }
    public void startCountdown(int start) {
        int count = start;
        while (count > 0) {
            System.out.println(count);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.err.println("Timer interrupted.");
            }
            count--;
        }
        System.out.println("Blast off!");
    }
}
```

```
public void startCountdown(int);
```

Code:

```
0: iload_1
1: istore_2
2: iload_2
3: ifle 22

6: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;
9: iload_2
10: invokevirtual #6 // Method java/io/PrintStream.println:(I)V
13: invokestatic #7 // Method java/lang/Thread.sleep:(J)V
16: iinc 2, -1
19: goto 2

22: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;
25: ldc #8 // String Blast off!
27: invokevirtual #6 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
30: return
```


Given this source, what is the bytecode?

```
public class CountdownTimer {  
    public static void main(String[] args) {  
        CountdownTimer timer = new CountdownTimer();  
        timer.startCountdown(10);  
    }  
    public void startCountdown(int start) {  
        int count = start;  
        while (count > 0) {  
            System.out.println(count);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                System.err.println("Timer interrupted.");  
            }  
            count--;  
        }  
        System.out.println("Blast off!");  
    }  
}
```

```
public CountdownTimer();  
Code:  
0: aload_0  
1: invokespecial #1 // Method java/lang/Object."<init>":()V  
4: return  
  
public static void main(java.lang.String[]);  
Code:  
0: new #2 // class CountdownTimer  
3: dup  
4: invokespecial #3 // Method "<init>":()V  
7: astore_1  
8: aload_1  
9: bipush 10  
11: invokevirtual #4 // Method startCountdown:(I)V  
14: return
```

Method Descriptors

- Specify the method's return type and parameter types
 - At a lower level than java source code
 - Notably does not include the method name
-
- `()I` – Describes a method that takes no parameters and returns an `int`.

Method Descriptors

1. Basic Types:

- `B` = `byte`
- `C` = `char`
- `D` = `double`
- `F` = `float`
- `I` = `int`
- `J` = `long`
- `L<classname>;` = an object of a class (e.g., `Ljava/lang/String;` for `String`)
- `S` = `short`
- `Z` = `boolean`

2. Void Return Type:

- `V` is used for methods that return `void`.

3. Array Types:

- For arrays, you use `[]` followed by the descriptor for the element type (e.g., `[I` for an array of `int`).

Examples:

Method with one **String** parameter and **void** return type:

- `(Ljava/lang/String;)V` – A method that takes a **String** and returns **void**.

Method with two parameters: **int** and **double**, and returns **boolean**:

- `(ID)Z` – A method that takes an **int** and a **double**, and returns a **boolean**.

Method with an array of **int** and a **String** parameter, returning **void**:

- `([ILjava/lang/String;)V` – A method that takes an array of **int** and a **String**, and returns **void**.

Method Descriptors - Match the method with its descriptor

```
1.  int foo(int x, int y)
2.  void bar(boolean myFlag)
3.  boolean isOpen(File f)
4.  String getName(Student s)
5.  int baz(Object o, int[] z)
```

```
a.  (Ljava/lang/Object;[I)I
b.  (Ljava/lang/String;)LStudent;
c.  ()V
d.  (Z)V
e.  (Ljava/io/File;)Z
f.  (II)I
g.  (I)Ljava/io/File;
h.  ()[I
i.  (LStudent;)Ljava/lang/String;
j.  ([I[I)V
k.  (Ljava/lang/Object;)V
```

Solution: 1f, 2d, 3e, 4i, 5a

Exercise 4

- Download Mystery2.class from Lec 4's reading
- Run javap to inspect the bytecode
- **Exercise:** Work through it to figure out:
 - What does this code do?
 - What do you think the source looked like?

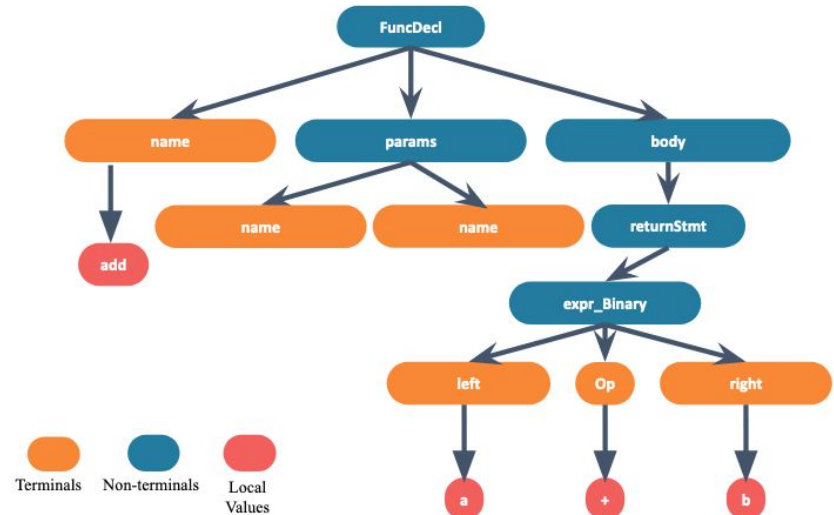
Program Representation 2: Abstract Syntax Trees

Representing Code as a Tree

Code has a natural hierarchical structure which cannot be captured as plain text

Classes have methods, methods have statements...

```
function add(a, b) { return a + b; }
```



AST Explorer

<https://astexplorer.net/> Make sure to switch the language to Java!

Answer the following questions:

1. What is the parent of the `name` identifier in the parameter of `addName`?
2. What is the parent of `methodDeclaration` for `addName`?
3. What is the parent of the `fieldDeclaration` for `addName`?
4. Change the field type from `ArrayList<String>` to `List<String>`. What changes?
5. Add a second parameter to `addName`. Where does it appear?

Java - Statements **vs** Expressions

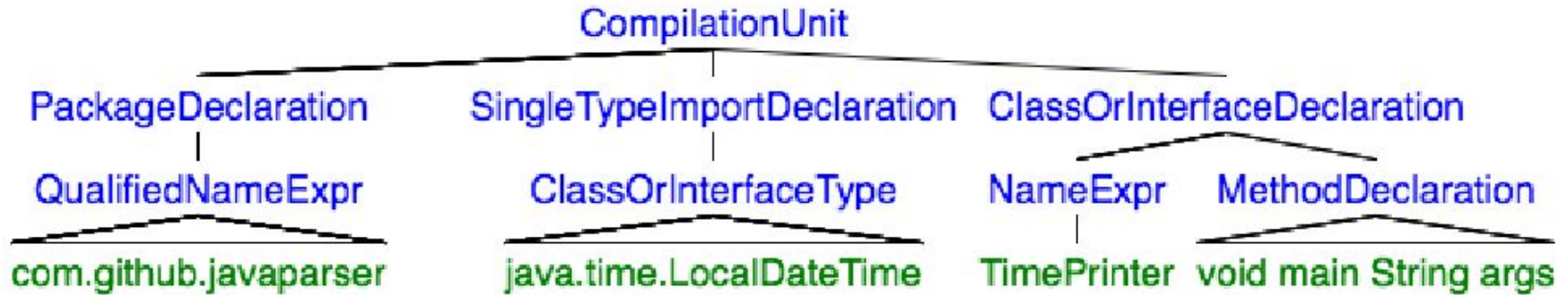
- A statement is a complete unit of execution that performs an action. It often ends with a semicolon
- An expression is a combination of variables operators, literals, and method calls that evaluates to a single value

Java - Statements **vs** Expressions

- `int x = 5;`
 - Statement
- `2 + 3`
 - Expr
- `System.out.println("Hello World");`
 - Statement
- `Math.sqrt(9)`
 - Expr
 - OR expression-statement

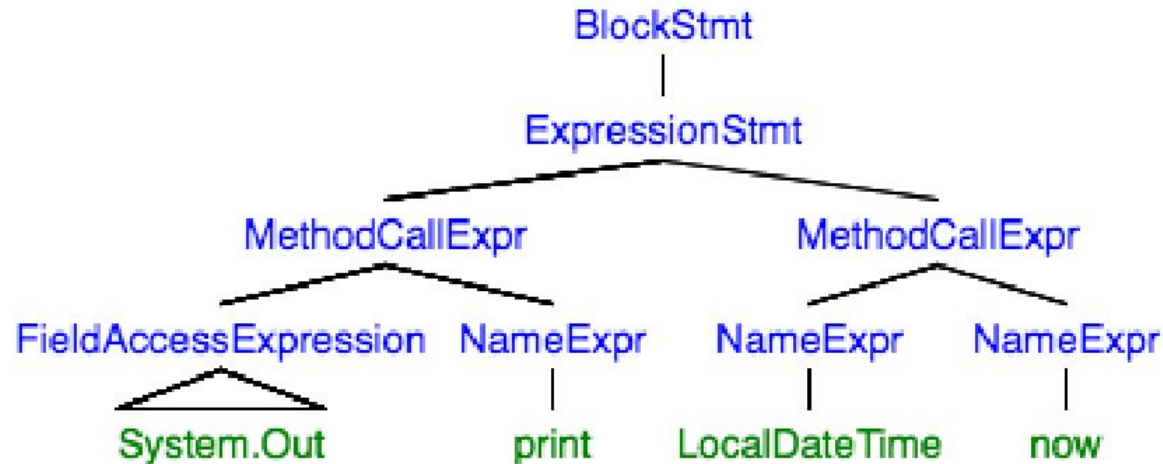
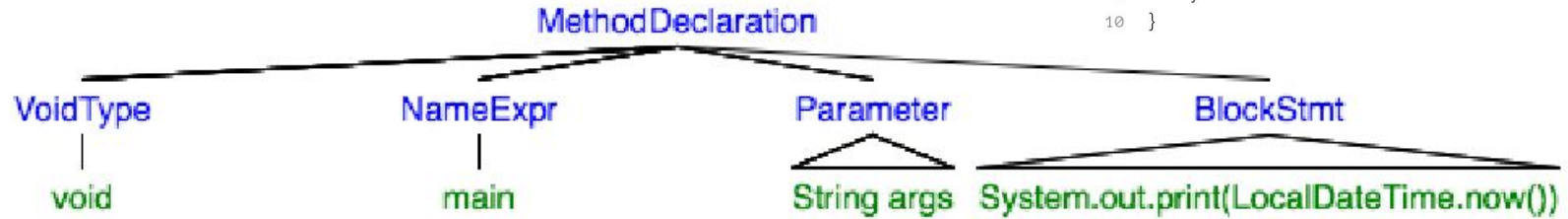
Java AST - node types

```
1 package com.github.javaparser;  
2  
3 import java.time.LocalDateTime;  
4  
5 public class TimePrinter {  
6  
7     public static void main(String args[]){  
8         System.out.print(LocalDateTime.now());  
9     }  
10 }
```



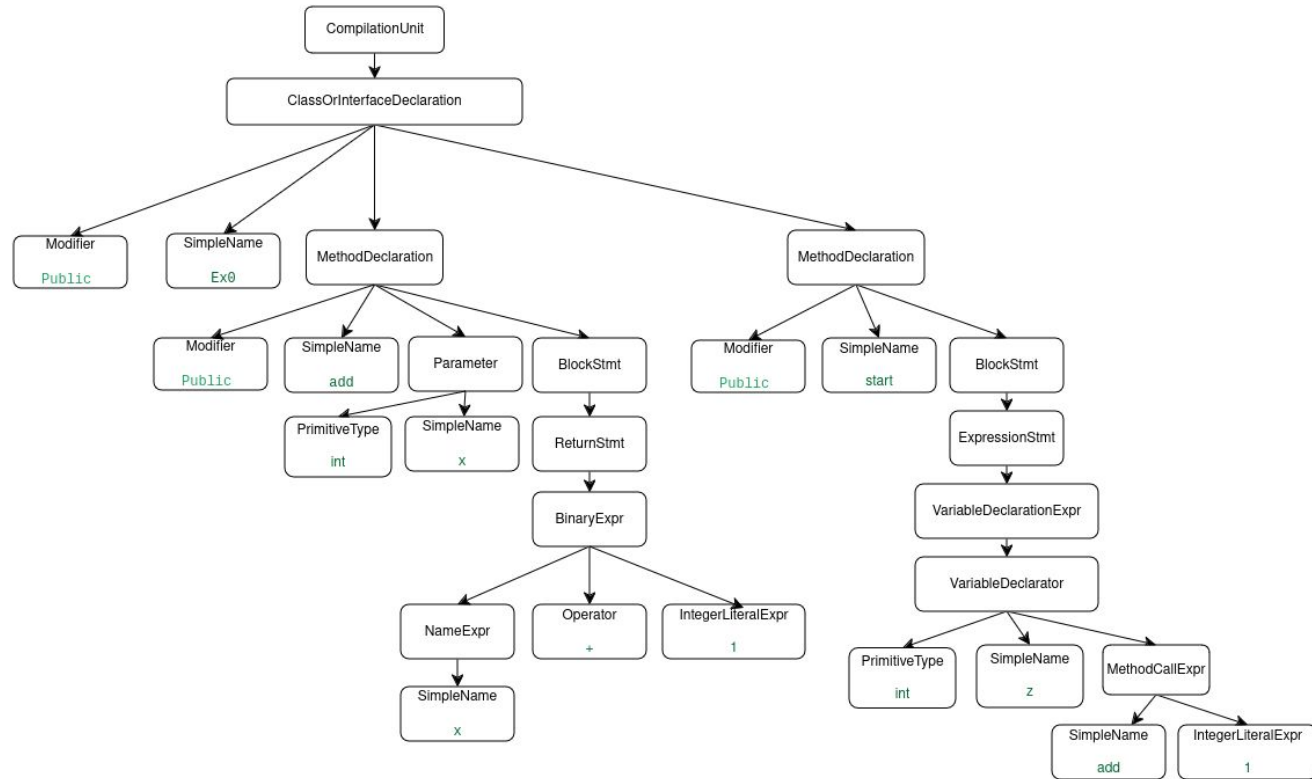
Java AST - node types

```
1 package com.github.javaparser;  
2  
3 import java.time.LocalDateTime;  
4  
5 public class TimePrinter {  
6  
7     public static void main(String args[]){  
8         System.out.print(LocalDateTime.now());  
9     }  
10 }
```



What would the AST look like?

```
public class Ex0 {  
  
    public int add(int x) {  
        return x + 1;  
    }  
  
    public void start() {  
        int z = add(1);  
    }  
}
```



Program Transformations

Why might we want to transform a program?

- Optimizations
- Obfuscations
- Debugging
- Performance Monitoring
- ...
- We'll learn more in this course!

AST Transformations

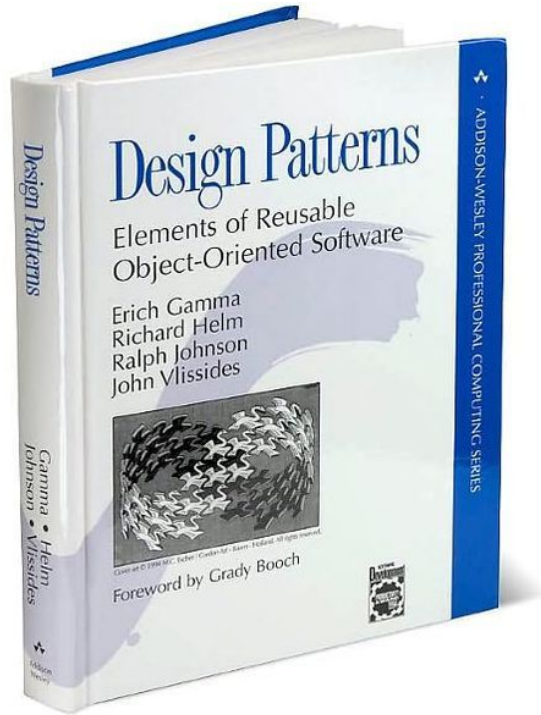
AST Transformations

Example: inject performance logging for each method (print time taken for each method)

- Find all method declarations
- Create new statements
 - Store start time
 - Store end time
 - Print end - start
- Insert them into the method declaration
 - In the proper location!

Traversing the AST tends to be laborious and error prone. This is largely due to the process relying on recursion and frequent type checking to attain your goal

Design Patterns



- Reusable solutions to common software design problems
- Provide a way to structure your code in a way that is flexible, maintainable, and efficient
- *“All well-structured object-oriented architectures are full of patterns. Indeed, one of the ways that I measure the quality of an object-oriented system is to judge whether or not its developers have paid careful attention to the common collaborations among its objects.”*

Visitor Design Pattern

- **Visitor Design Pattern** commonly used for tree transformations
- Allows you to separate the operations / transformations from the traversal process
- Trees often have different types of nodes. The Visitor pattern makes it easy to define different operations for each node type by using method overloading in the visitor.
- Each operation is encapsulated in a "visitor," which visits different node types (like addition or multiplication) and processes them.

JavaParser Library

- Allows you to interact with Java source code as an AST
- Provides mechanism to navigate the tree with *Visitor Support*
 - Allows you to write operations without needing to write traversal code

A Simple Visitor

Let's create a visitor to print the name of each method in a program

You can also pass a value to the visitor!

- Useful to save or load state

A Simple Modifying Visitor

Let's write a visitor to change variable names (declarations and references) to all caps in Method `decls`

```
public class Vars {  
  
    public void decls() {  
        int foo = 15;  
        int helloworld = 20;  
        int added = foo + helloworld;  
    }  
  
    public void dontchange() {  
        int lower = 15;  
        int stillLower = 10;  
        int lowercase = lower + stillLower;  
    }  
}
```

JavaParser library

Basic Workflow:

1. Parse the source into an AST
 - a. `JavaParser.parse(sourceCode);`
2. Transform the AST
 - a. Using a Visitor, traverse the AST and perform a transformation
3. Write the modified AST
 - a. Call `toString()` on the AST root (`CompilationUnit`) to get the source

Bytecode Transformations

Review: Java Bytecode

```
public class Ex0 {  
  
    public int add(int x) {  
        return x + 1;  
    }  
  
    public void start() {  
        int z = add(1);  
    }  
}
```

public int add(int);
Code:
0: iload_1
1: iconst_1
2: iadd
3: ireturn

LocalVariableTable:
Start Length Slot Name
0 4 0 this LEx0;
0 4 1 x I

public void start();
Code:
0: aload_0
1: iconst_1
2: invokevirtual #2 // Method add:(I)
5: istore_1
6: return

LocalVariableTable:
Start Length Slot Name
0 7 0 this LEx0;
6 1 1 z I

Operand Stack



Local Variable Table



Example 2

```
public class Ex1 {  
  
    public void foo(int x) {  
        int y = x + 1;  
    }  
  
    public void baz() {  
        int z = 99;  
        foo(z);  
    }  
}
```

```
public void foo(int);
```

Code:

```
0: iload_1  
1: iconst_1  
2: iadd  
3: istore_2  
4: return
```

LocalVariableTable:

Start	Length	Slot	Name
0	5	0	this LEx1;
0	5	1	x I
4	1	2	y I

```
public void baz();
```

Code:

```
0: bipush      99  
2: istore_1  
3: aload_0  
4: iload_1  
5: invokevirtual #2 // Method foo:(I)V  
8: return
```

LocalVariableTable:

Start	Length	Slot	Name
0	9	0	this LEx1;
3	6	1	z I

Operand Stack



Local Variable Table



ASM

Java Library for Bytecode manipulation

Basic Workflow:

1. Read the bytecode
 - a. `ClassReader`
2. Transform the bytecode
3. Write the modified bytecode
 - a. `ClassWriter`



Bytecode Transformation

Increment every constant by 1 in the main method

```
public class Constants {  
  
    public static void dontModify(int q) {  
        int x = 100;  
        int y = 10;  
        int z = x + y + 1;  
    }  
  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
        int c = 30;  
  
        dontModify(100);  
    }  
}
```

```
public static void dontModify(int);
```

Code:

```
0: bipush      100  
2: istore_1  
3: bipush      10  
5: istore_2  
6: iload_1  
7: iload_2  
8: iadd  
9: iconst_1  
10: iadd  
11: istore_3  
12: return
```

```
public static void main(java.lang.String[]);
```

Code:

```
0: bipush      10  
2: istore_1  
3: bipush      20  
5: istore_2  
6: bipush      30  
8: istore_3  
9: bipush      100  
11: invokestatic #7    // Method dontModify:(I)V  
14: return
```

Summary

- Java bytecode method invocations
- We learned how to transform programs over two different representations
- HW2 Released
 - AST and Bytecode transformations